LISTAS



Structs e Arrays

Uma **STRUCT** é um tipo de valor que encapsula os dados relacionados juntos, facilitando o gerenciamento e a manipulação:

```
struct Estudante {
    var nome: String
    var idade: Int
}
```

- Construindo OBJETOS ou ARRAY DE OBJETOS:

```
var aluno: Estudante = Estudante(nome: "João", idade: 25)

var alunos: [Estudante] = [
    Estudante(nome: "João", idade: 25),
    Estudante(nome: "Maria", idade: 23)
]
```

HACKA TRUCK Maker Space

List

Estrutura que permite iterar sobre uma coleção, como um *array*, e criar uma visualização para **cada elemento** dessa coleção já com um **estilo predefinido** pela linguagem.

```
NavigationStack {
    VStack{
        List(estudantes, id: \.self) { e in
            Text(e.nome)

        }.navigationTitle("Alunos")
        .toolbar {
            Button("Adicionar Aluno") {
                estudantes.append(Estudante(nome: "Novo Aluno", idade: 27))
            }
        }
    }
}
```



Não permite muitas alterações de estilo e tem ScrollView acoplado.

HACKA TRUCK Maker Space

ForEach

Estrutura que permite iterar sobre uma coleção, como um *array*, e criar uma visualização para cada elemento dessa coleção com um estilo **personalizado**.

```
NavigationStack {
    VStack {
        ForEach(estudantes, id: \.self) { e in
            Text(e.nome)
                .padding(10)
                .frame(minWidth: 100, maxWidth: 40)
                .background(.gray)
                .foregroundStyle(.white)
        }
    }
    .navigationTitle("Alunos")
    .toolbar {
        Button("Adicionar Aluno") {
            estudantes.append(Estudante(nome: "João", idade: 10))
        }
    }
}
```



Permite alterações de estilo e não tem ScrollView acoplado.

HACKA TRUCK Maker Space

Listas

```
var alunos: [Estudante] = [
    Estudante(nome: "Hermione", idade: 11),
    Estudante(nome: "Harry Potter", idade: 10),
    Estudante(nome: "Rony", idade: 12)
]
```

[]

```
ForEach( [  ] ) { aluno in
    Text(aluno.nome)
}
```

1ª Iteração - personagem = 

2ª Iteração - personagem = 

3ª Iteração - personagem = 

HACKA TRUCK Maker Space

Elementos da Lista - Identificador Único

É importante que sejam identificados de forma única para que, quando sejam “clicados”, consigam informar o **código** de quem foi que recebeu o clique.

- A sintaxe `id: \.CAMPO` é usada para identificar exclusivamente cada elemento na lista a partir de um campo da própria Struct (ex: **\.nome** ou **\.idade**).

```
struct Estudante {
    var nome: String
    var idade: Int
}
```

- A sintaxe `id: \.self` é usada para “burlar” a identificação única de cada elemento da lista no caso de não acessarmos outra tela a partir da lista.

HACKA TRUCK Maker Space

Elementos da Lista - Identificador Único

É importante que sejam identificados de forma única para que, quando sejam “clicados”, consigam informar o **código** de quem foi que recebeu o clique.

	id: \.campo	id: \.self
O que é o **identificador**?	Uma propriedade específica do objeto	O hash do objeto
Uso ideal:	Arrays de String, Int, Enum	Arrays de Structs ou Classes
Duplicatas:	Permite dados iguais, desde que haja um campo diferente	Não permite valores iguais (causa bug)

HACKA TRUCK Maker Space

Hashable *vs* Identifiable

```
struct Estudante: Hashable {
    let nome: String
    let idade: Int
}
```

Protocolo **Hashable**: por meio de um cálculo matemático, gera cada filho da Struct de forma única, criando um *hash*.

```
List(alunos, id: \.self) { aluno in
    Text(aluno.nome)
}
```

```
ForEach(alunos, id: \.self) { aluno in
    Text(aluno.nome)
}
```

Uma vez com Hashable, tanto List quanto ForEach reconhecem o ID por meio do \.self.

```
struct Estudante: Identifiable {
    let id: Int
    let nome: String
    let idade: Int
}
```

Protocolo **Identifiable**: obriga que tenhamos um campo chamado id na Struct para servir de identificador.

```
List(alunos) { aluno in
    Text(aluno.nome)
}
```

```
ForEach(alunos) { aluno in
    Text(aluno.nome)
}
```

Uma vez com Identifiable, tanto List quanto ForEach reconhecem de forma automática o ID.

HACKA TRUCK Maker Space

Desafio:

```
struct Conteudo: Hashable {
    let categoria: String
    let nome: String
    let capa: String
    let ano: String
    let genero: String
    let pais: String
    let imdb: Double
}
```

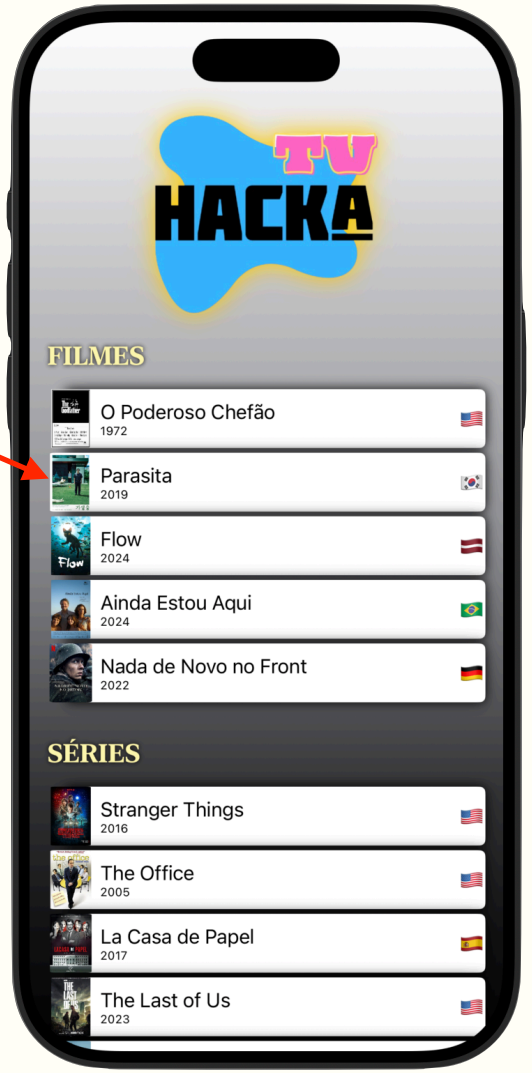
```
@State var arrayConteudos: [Conteudo] = [
    Conteudo(categoria: "Série", nome: "Stranger Things", capa:
        "https://upload.wikimedia
        .org/wikipedia/pt/2/29/Stranger_Things_Temporada_1_Poster.jpg",
        ano: "2016", genero: "Terror", pais: "🇺🇸", imdb: 8.6),
    Conteudo(categoria: "Série", nome: "The Office", capa:
        "https://upload.wikimedia
        .org/wikipedia/pt/2/2a/The_Office_temporada_1.jpg", ano: "2005",
        genero: "Sitcom", pais: "🇺🇸", imdb: 9),
```

...

HACKA TRUCK Maker Space

Desafio:

AsyncImage
com Placeholder





ScrollView horizontal



HACKA TRUCK Maker Space



/hackatruck-makerspace



@hackatruck_makerspace



@hackatruck_makerspace



www.hackatruck.com.br